

Knowledge Studio

How to turn your documents into a searchable knowledge base your workflows can ask questions of.

Eurskem AI Platform · Practical guide

What it does

You upload documents. Knowledge Studio reads them, splits them into passages, converts each passage into a vector so it can be searched by meaning, and stores the result. A workflow can then ask a plain-language question and get an answer grounded in your documents, with citations back to the source.

Six things, in order

Collection	What knowledge exists. A named container for a set of documents — for example "Dura 25 Product Knowledge".
Profiles	How documents get prepared: which parser reads them, how they are split, which embedding model is used.
Index	The searchable result of one ingestion run. A Collection can hold several; exactly one is active and gets searched.
Retrieval Profile	How to search: keyword, meaning, or both; how many passages to consider and return.
RAG Agent	A saved bundle of Collection + Retrieval Profile + answering model. This is what a workflow points at.
Workflow	Why and when it is used, in the Builder.

The bar at the top is the important part. Every tab in Knowledge Studio acts on the one Collection named in the bar — it shows the name, the ID, how many documents and passages it holds, and which Index is actually being searched. Change the Collection there, and every tab follows.

The short version

- 1 Collections** — create one. Give it a clear name.
- 2 Ingestion** — pick a preset, choose your files, run it. Wait for "completed".
- 3 Documents & Indexes** — click Activate on the new index. Until you do, the Collection is not searchable.
- 4 Retrieval Playground** — ask a real question and check the passages that come back. Save the settings that work as a Retrieval Profile.

- 5 **Profiles & RAG Agents** — create a RAG Agent, test it, copy its `rag_agent_id`.
- 6 **Builder** — drop in a RAGAgent node, paste the ID, map your question to it.

Step 1 — Create a Collection

Open **Knowledge Studio** → **Collections**. Give it a name and, optionally, document types.

Document types matter more than they look: they are one of the signals used to pick an embedding model automatically. Marking a Collection as `manual`, `spec`, `policy` or `contract` tells the system this is precision-sensitive material where a wrong passage is expensive, and it selects a higher-quality model.

Once created, the Collection becomes the one you are "working in". Its ID is shown in the bar — click it to copy.

Step 2 — Ingest documents

Open **Ingestion**. Confirm the bar shows the right Collection, then choose a preset.

Presets

- **Technical Documentation** — manuals, datasheets, engineering docs. Retrieves small precise passages, then expands to the surrounding section for context.
- **General Documents** — reports, notes, mixed prose.
- **Policies / Contracts** — keeps clauses and headings intact.
- **Fast Demo** — smaller passages, quickest to run.

Parser — how the file is read

- **Standard** — plain text extraction. Right for most files.
- **Layout-aware** — keeps pages, slides, sheets and headings as boundaries.
- **Structure-aware** — as above, with tidier section handling.
- **Vision-augmented** — for PDFs whose meaning lives in pictures: pump curves, wiring diagrams, compatibility matrices saved as images. A vision model reads each such page and writes out what it shows, so the content becomes searchable text.
- **OCR fallback** — for scanned PDFs with no text layer at all.

When to reach for Vision-augmented. If a fact only appears inside a chart or a picture of a table, normal extraction cannot see it and no amount of search tuning will find it. Vision-augmented costs more and runs slower, so use it for the documents that need it rather than everything.

Embedding model

Leave it on **Auto** unless you have a reason not to. Auto reads the Collection's document types, its size and its language, then picks:

- precision-sensitive types (manual, spec, policy, contract) → the highest-quality model;
- substantial non-Latin text → a multilingual model;

- large corpora → the faster, cheaper model, because quality gains rarely repay the cost at that scale;
- everything else → the balanced default.

Whatever it picks is recorded on the Index along with the reason, so an old run stays explainable later.

Chunking

The preset sets this. Change it only if results disappoint:

- **recursive** — sensible default for prose.
- **parent_child** — searches small passages, answers with the larger section around them. Good for manuals.
- **sentence_window** — very precise matching; best for lookups like a single tolerance or part number.
- **structure_aware** — respects headings; good for contracts.
- **contextual** — prefixes each passage with its document and section, which helps when passages read ambiguously alone.

Files

PDF, DOCX, XLSX, PPTX, CSV, TSV, TXT, Markdown, JSON, HTML, XML and common code files. Up to **70 MB per file**, 20 files per run.

Press Run ingestion and watch the progress bar. The stages are: uploading → parsing → chunking → enriching → embedding → indexing.

If a document fails, the job tells you why, per file. The two common messages:

"no text could be extracted" — the file has no text layer. Re-run it with the Vision-augmented or OCR fallback parser.

"produced no searchable chunks" — the chunking settings do not fit the document. Lower the target token size, or use a different strategy.

Step 3 — Activate the Index

Open **Documents & Indexes**. Each ingestion run produced an Index. Press **Activate** on the one you want searched.

This is the step people miss. Until an Index is active, the Collection has documents but answers nothing — the bar will say "none — activate an index to make this collection searchable".

Because workflows point at a Collection rather than an Index, you can re-ingest with better settings, activate the new Index, and every workflow picks it up with no edits. If the new one is worse, activate the old one again. Old runs still record which Index they actually used.

The same tab lists every document with its status and, when something went wrong, its error.

Step 4 — Test retrieval before trusting it

Open **Retrieval Playground** and ask a question you already know the answer to. This is the cheapest way to find out whether the knowledge base is any good, and it uses no answering model at all — you are looking at the raw passages.

Strategies

- **dense** — meaning-based. Finds "PTFE resists sulphuric acid" from "what seal survives acid?" even with no shared words.
- **sparse** — keyword. Best for exact part numbers and codes.
- **hybrid** — both together. The right default.
- **hybrid_rerank** — hybrid, then a model re-orders the results for relevance. Slower, usually the most accurate.

Use Compare Dense vs Hybrid vs Reranker to run all three side by side on the same question and see which actually retrieves the passage you wanted.

If results are poor

- Right passage ranked too low → turn on reranking, or raise the candidate count.
- Nothing relevant at all → the content may not be in the index; check Documents, and consider Vision-augmented parsing.
- Passages are fragments that read out of context → try `parent_child` or `contextual` chunking and re-ingest.
- Too much noise → lower the final count.

When a configuration works, press **Save experiment as profile**. That saved Retrieval Profile is what a RAG Agent will use.

Step 5 — Create a RAG Agent

Open **Profiles & RAG Agents**. Name it after the job it does — "Verder Product Support" — and choose:

- the **Retrieval Profile** you just saved;
- the **answering model**. Leave it on Best possible LLM (Auto) unless you need a specific one; Auto picks deterministically per call.

Test it there with a real question. You should get an answer with citations. If the answer is right but the citations look wrong, go back to the Playground — the retrieval is the problem, not the model.

Then copy the `rag_agent_id`.

Step 6 — Use it in a workflow

In the Builder, add a **RAGAgent** node, paste the ID, and map the question to it:

```
- id: product_knowledge
  type: RAGAgent
  config:
    rag_agent_id: rag_01J007
    query: "{{inputs.question}}"
```

To narrow results at run time — without ever widening what the user is allowed to see:

```
runtime_filters:
  product: "{{outputs.extract.product}}"
```

If you want passages but not a generated answer, use **KnowledgeRetrieval** instead. It returns the passages, citations and assembled context for a later node to work with.

Validation before you run. Preflight checks that the RAG Agent exists, is active, and that its Collection has an active Index — before spending a single token. If you deleted the agent or never activated an index, it says so up front instead of failing halfway through.

Reading the results

After a run, the Cockpit shows which RAG Agent answered, which Collection and Index it resolved, how many passages were considered and used, the latency and cost, and the citations. Open Retrieval Trace shows the exact passages that produced that answer.

Traces record the exact resources resolved at the time, so reopening an old run tells you what really happened, even if the Collection's active Index has changed since.

Citations are sources, not proof. A RAG citation means "this passage was retrieved and used" — it is not verified evidence. Proposal-grade and scientific work must keep using the evidence-verification pipeline, which independently checks each claim against its source.

Fixing common problems

"I can't upload anything"

Check the bar. With no Collection, there is nothing to upload into and the button stays disabled. Create one first.

"It finds nothing"

Almost always no active Index. Go to Documents & Indexes and press Activate.

"It can't answer questions about a chart or a diagram"

Normal parsing does not see inside images. Re-ingest that document with the Vision-augmented parser, then activate the new Index.

"Answers cite the wrong passage"

Test the same question in the Playground. If retrieval brings back the wrong passages, fix retrieval — reranking, or different chunking — rather than changing the answering model.

"It got slow, or I see rate-limit errors"

The API allows 60 requests per minute per user. Several browser tabs polling at once can reach that. Close the spare tabs.

Rules worth remembering

- An Index is immutable. To change how documents are prepared, ingest again and activate the new Index.
- Changing the embedding model means a new Index — vectors from different models are not comparable.
- Workflows reference a RAG Agent, never an Index. That is what lets you improve the knowledge base without editing workflows.
- Runtime filters can only narrow results. Nothing supplied at run time can widen what a user is allowed to see.
- Test retrieval in the Playground before blaming the model.